TRACE-GRAG: Retrieval Recall Gates Bug Localization by a Local Gemma 4 Agent

Lexical, semantic & graph retrieval feeding a local Gemma 4 (E2B) bug-localization agent: a CI-backed study on consumer hardware

Bobby Nandigam. Submission to the Google — Gemma 4 Developer Agent **Paper Track** (2026). Topic: *Code Comprehension — code-graph generation, parsing, and embedding*.

> Every number in this paper is produced by the committed, released evaluation harness (retrieval run `run-20261112-103442`; Gemma run `gemma-gemma4_e2b-20261112-121138`), not hand-entered. All code, the data manifest, and the harness are released (Apache-2.0) for verification and reuse. The agent runs **Gemma 4 E2B locally** on a 16 GB Apple-Silicon Mac.

Abstract

We ask whether better repository retrieval helps a **locally-run Gemma 4 (E2B)** agent localize real bugs, on 24 SWE-bench Lite tasks under consumer-hardware constraints (a 16 GB Apple-Silicon Mac, no SWE-bench Docker harness). Three retrieval conditions — lexical (BM25), semantic (dense `nomic-embed-text`), and graph (lexical-seeded expansion over a provenance-disciplined code graph) — each supply the top-10 candidate symbols to Gemma 4, which picks the one to fix. Reporting every retrieval metric with 95% bootstrap confidence intervals and the Gemma decision on all 24 tasks, we find: (1) **retrieval recall gates the agent** — semantic retrieval's higher recall@10 (0.71 vs 0.58) yields the best Gemma localization (0.46 vs 0.38); (2) **Gemma reranking nearly doubles top-1 localization** over trusting the retriever's own rank-1 (0.38 vs 0.21 for lexical), i.e. a small local model reading ten candidates substantially repairs weak retrieval ranking; (3) given the gold symbol is in the top-10, Gemma picks it ~64% of the time **regardless of which retriever supplied the candidates**, so the retriever's job is recall; and (4) lexically-seeded **graph expansion adds nothing** — it is identical to BM25 for both retrieval and the agent — while semantic retrieval costs ~500× more latency locally (median 47 s vs 0.09 s). For on-device SE agents, the lesson is to spend the compute budget on retrieval *recall* (and candidate reranking by the model), not graph expansion. All code and the harness are released (Apache-2.0).

1. Introduction

Large language models can resolve real GitHub issues (SWE-bench [2]), whether through tool-using agents (SWE-agent [3]) or simpler localize-then-repair pipelines (Agentless [1]). A consistent finding is that **finding the right code to change** — localization — is decisive for downstream success and is cheap to study in isolation. Separately, a growing line of work argues that **repository-level code graphs** improve these systems: RepoGraph [4] plugs a line-level dependency graph into four SWE-bench methods and improves them.

Two things are under-examined. First, most results come from large hosted models with the full SWE-bench Docker evaluation harness (>100 GB of images). The regime of a **solo developer or an on-device Gemma-class agent** — a 16 GB machine, tight disk, no Docker harness — is different and under-reported. Second, the literature foregrounds **where graphs help** and rarely isolates **where they do not**, with calibrated uncertainty.

This paper contributes a controlled, reproducible study in exactly that constrained regime. We build a **provenance-disciplined** repository code graph that never fabricates an edge, and we compare three retrieval conditions for bug localization on identical graphs: **E1 lexical (BM25)**, **E2 semantic (dense embeddings)**, and **E3 graph (lexical-seeded graph propagation)**. We report hit@k, recall@k, and MRR with 95% bootstrap confidence intervals, on a disjoint dev/eval split of 24 SWE-bench Lite tasks across 10 repositories.

Our contributions are: (1) a measured study of **retrieval-conditioned localization by a local Gemma 4 E2B agent**, showing that retrieval *recall* gates the agent, that model reranking of ten candidates nearly doubles top-1 localization over raw retrieval, and that the model's picking skill is retriever-agnostic; (2) an auditable, provenance-disciplined code-graph construction that never fabricates an edge; and (3) an honest retrieval comparison with bootstrap CIs showing that graph expansion adds nothing over BM25 while semantic retrieval buys recall at ~500× the latency. We do **not** claim that using a graph for repository understanding is novel (RepoGraph [4] and others precede us); our contribution is measured evidence, on consumer hardware with the competition's own Gemma 4 model, about *what actually helps a small local agent localize bugs*.

2. Problem Definition

Given a repository `R` at a fixed base commit and a natural-language issue `q` (the `problem_statement`), **bug localization** ranks the code symbols of `R` by their relevance to fixing `q`. A symbol is a FUNCTION, METHOD, or CLASS with a precise source span. The **gold** set is derived from the human fix: the symbols whose base-commit line spans intersect the patch's changed (minus-side) lines, excluding test files (we score finding the *implementation* to fix, not the hidden tests). Quality is measured by:

- **hit@k** — 1 if any gold symbol is in the top-k, else 0;
- **recall@k** — fraction of gold symbols in the top-k;
- **MRR** — reciprocal rank of the first gold symbol.

The agent never sees the gold patch or the hidden tests; gold is evaluator-only.

3. Related Work

Localize-then-repair [1] and agentic [3] approaches both depend on localization [2]; code graphs from Code Property Graphs [5] to RepoGraph [4] add repository structure; lexical [9] and dense [8,10,11] retrieval are the standard baselines; and Graph RAG [7] is the natural-language ancestor of code-graph retrieval. **RepoGraph [4] is the closest prior art** — it shows a line-level dependency graph improves four SWE-bench systems. This work differs by *isolating retrieval quality for localization under local-inference constraints* and reporting **negative as well as positive** results with confidence intervals. We do not claim graph-for-repository-understanding as novel; we contribute measured evidence of *when* it helps.

4. Proposed Architecture

TRACE-GRAG is a seven-stage pipeline; stages 1–5 and the evaluation are implemented and measured here, stages 6–7 are scaffolded future work (§11).

```
issue → [1] Ingestion → [2] Parsing+Graph → [3] Retrieval → [4] Ranking
       → [5] Compression → [6] Agent (Gemma) → [7] Execution (Tier-B pilot)
```

Code graph (stage 2). A two-pass `ast` parser emits six node types (FILE, MODULE, CLASS, FUNCTION, METHOD, TEST) and six edge types (CONTAINS, DEFINES, IMPORTS, CALLS, TESTS, COVERS). Each node carries a stable id, repo-relative path, line span, content hash, signature, and docstring. **Every edge carries a `Provenance`** (`ast_static`, `import_resolved`, `call_resolved`, `name_heuristic`, `coverage_dynamic`, `unresolved`) and a calibrated confidence. **Unresolvable call/import targets are recorded as `unresolved`, never invented** — a property enforced by a unit test. The graph is a NetworkX MultiDiGraph with version-independent flat-JSON persistence.

Retrieval (stage 3). Three conditions share the same candidate set (FUNCTION/METHOD/ CLASS nodes) and the same issue text: - **E1 — BM25** over a per-symbol document (qualified name, name, signature, docstring), with snake/camel identifier tokenization. - **E2 — Semantic**: cosine similarity between the issue embedding and per-symbol embeddings (`nomic-embed-text`, 768-dim, served locally by Ollama); embeddings are cached by content hash for reproducibility. - **E3 — Graph**: BM25 seeds the top symbols, then structural relevance propagates over the undirected code graph with per-hop decay; a node's score mixes propagated structural mass with its own lexical score.

Agent (stage 6). The retriever's top-K candidate symbols (name, signature, truncated docstring) are presented to a local **Gemma 4 E2B** model, which selects the single symbol to fix. This reduces the agentic core to one bounded, verifiable decision and is **evaluated here** (§7.4). An unparseable answer is recorded as an abstention, never guessed into a hit.

Ranking, compression, full repair loop, execution (stages 4,5,7). The design further targets a combined score `S(v,q)=αL+βE+γT+δG` with a **test-evidence term** `T`, budget-aware connected-subgraph selection, a multi-turn repair loop, and per-repo venv sandboxed test execution. These remain future work (§11); no result here depends on them.

5. Methodology

Graph integrity. Confidence follows provenance (`ast_static`=1.0, `import_resolved`=0.95, `call_resolved`=0.9, `name_heuristic`=0.6); unresolved targets are kept at 0.3 with their raw name rather than asserted as edges. Call resolution is bare-name and may over-connect same-named symbols (§10).

Gold construction. We parse each gold patch's diff, take the **minus-side** (base-commit) line ranges of changed non-test files, and map them to the FUNCTION/METHOD/CLASS nodes whose spans intersect — evaluator-only and deterministic.

Statistics. Metrics are means over tasks with a **95% bootstrap CI** (2000 resamples, fixed seed). All retrieval conditions are untrained with fixed defaults, so there is no split leakage.

6. Experimental Setup

- **Benchmark:** 24 SWE-bench Lite tasks across 10 repositories, deterministically selected (`scripts/build_manifest.py`), disjoint **6 dev / 18 eval**. Graph sizes range from ~850 nodes (requests) to ~25.9k (sympy).
- **Hardware:** Mac mini **M4, 16 GB** unified memory, macOS 26.3, arm64; ~22 GB free disk.
- **Models/runtimes:** all local via Ollama 0.40.16 — `nomic-embed-text` (E2 embeddings) and `gemma4:e2b` (the agent; ~7.5 GB, loads in 16 GB RAM). Gemma runs with thinking disabled and temperature 0 for a deterministic, parseable single-symbol choice (Gemma 4 is a reasoning model; left on, its tokens go to a separate thinking channel and the answer overruns the budget).
- **Procedure:** for each task, fetch a disk-safe tarball snapshot at the base commit, build the graph, derive gold symbols, run E1/E2/E3, then give each retriever's **top-10** candidate symbols to Gemma

4 and record its pick; score; delete the snapshot (peak disk = one repo). Gemma never sees the gold patch or hidden tests. Full commands in the released repository.

7. Results

All 24 tasks completed with **0 errors**; E1, E2, and E3 ran on every task. Table 1 reports the full set (mean with 95% bootstrap CI); the disjoint dev (n=6) and eval (n=18) splits show the same pattern (`results/summary.json`). Latency is the harness mean; for E2 we also give the median and max, since it is right-skewed by the largest graphs.

Table 1 — Localization on 24 SWE-bench Lite tasks. mean [95% bootstrap CI].

strategy	hit@1	hit@5	hit@10	recall@10	MRR	lat (s)
E1 BM25	0.208 [0.042,0.375]	0.500 [0.292,0.708]	0.583 [0.375,0.750]	0.376 [0.228,0.538]	0.310 [0.167,0.473]	0.13
E2 Semantic	0.333 [0.167,0.542]	0.417 [0.208,0.625]	0.542 [0.333,0.708]	0.336 [0.192,0.481]	0.405 [0.235,0.586]	67.2 (med 46.9, max 212.8)
E3 Graph	0.208 [0.042,0.375]	0.500 [0.292,0.708]	0.583 [0.375,0.750]	0.376 [0.228,0.538]	0.310 [0.169,0.471]	0.18

On the held-out **eval** split (n=18) the ordering is unchanged: MRR 0.334 (E1) / 0.415 (E2) / 0.332 (E3); hit@1 0.222 / 0.333 / 0.222.

F1 — Graph expansion yields no net benefit over lexical. E3 equals E1 *exactly* in aggregate on hit@1/5/10 and recall@10, and to three decimals on MRR (0.310). Per task, graph expansion *does* change the ranking (it alters MRR on 10/24 tasks, mostly large repos) — but the gains and losses cancel, leaving aggregate quality unchanged at higher latency. Static, lexically-seeded graph expansion is therefore not justified for localization on this set.

F2 — Semantic retrieval is complementary, not dominant. E2 lifts hit@1 (0.333 vs 0.208) and MRR (0.405 vs 0.310) and **recovers repos lexical misses** (per-repo hit@10: xarray 1.00 vs 0.50, astropy 0.50 vs 0.00) but **regresses where lexical wins** (scikit-learn 0.33 vs 1.00). Head-to-head MRR: 12 wins / 9 losses / 3 ties; **all CIs overlap — no significant winner at n=24.**

F3 — Semantic retrieval is ~500× more expensive locally. Median E2 latency is 46.9 s per issue (max 212.8 s on sympy's ~26k-node graph) versus 0.09 s median for BM25, because every candidate symbol must be embedded through the local model. For an interactive on-device agent this cost is frequently prohibitive.

7.4 Gemma 4 in the loop — does retrieval help the agent localize?

We now give each retriever's **top-10** candidate symbols to the local **Gemma 4 E2B** agent and record the single symbol it picks (run `gemma-gemma4_e2b-20261112-121138`, 24 tasks, 0 errors, 0 abstentions). We report three rates: `gemma_hit@1` (the agent's pick is gold — the headline), `gold_in_top10` (the recall gate — the agent cannot pick what retrieval never surfaced), and `retr_top1` (the retriever's own rank-1, the no-agent baseline).

Table 2 — Gemma 4 E2B localization, by the retriever that fed it (rate over 24 tasks).

feeding retriever	gemma_hit@1	gold_in_top10	retriever top-1 (baseline)	pick-rate given gold in top-10
E1 BM25	0.375	0.583	0.208	0.643
E2 Semantic	**0.458**	**0.708**	0.417	0.647
E3 Graph	0.375	0.583	0.208	0.643

F4 — Retrieval recall gates the agent. The agent's accuracy tracks `gold_in_top10`: semantic retrieval's higher recall (0.708 vs 0.583) is exactly what lifts Gemma's localization (0.458 vs 0.375). Getting the gold symbol *into* the candidate set is the retriever's decisive job.

F5 — Gemma reranking nearly doubles top-1 localization. Reading ten candidates and picking one, Gemma reaches 0.375 where BM25's own rank-1 was 0.208 (and graph likewise 0.208 → 0.375) — a large gain from a 2.3B-effective-parameter model running locally. Even for semantic, Gemma (0.458) edges its already-strong rank-1 (0.417). A small local model repairs weak retrieval ranking.

F6 — The agent's picking skill is retriever-agnostic. Conditioned on the gold symbol being present in the ten, Gemma selects it ~64% of the time whatever the source (0.643 / 0.647 / 0.643). So the three conditions differ in the agent's end accuracy *only* through recall (F4), not through any interaction with the candidate ordering — and graph expansion, which does not change recall, again changes nothing (E3 = E1 exactly).

8. Ablation Studies

We report no fabricated ablation rows. The graph condition's parameters (hop depth, seed count, decay) were held fixed; given F1 — graph expansion nets to zero — the informative ablation is the test-evidence term and combined ranker `S=αL+βE+γT+δG` (future work, §11). The `top-K` budget (fixed at 10) and E4B vs E2B are the other natural sweeps.

9. Discussion

Complementarity and why graph expansion fails. Lexical and semantic retrieval win *disjoint* repositories (F2): lexical nails scikit-learn/pylint; semantic recovers xarray/astropy where lexical scores zero — motivating fusion. Graph expansion, seeded from lexical matches, inherits the seed's errors and cannot surface a gold symbol the seed never retrieved (hence F1's net-zero effect), so it is *not* the path to improvement.

What actually helps the agent (F4–F6). The clearest, most actionable result is that a small local Gemma 4 benefits from retrieval through **recall**, not ranking or structure: pushing the gold symbol into the top-10 (where semantic retrieval wins) is what raises agent accuracy, and the model then recovers it ~64% of the time on its own, nearly doubling top-1 over raw retrieval. For a practitioner building an on-device SE agent, the budget is best spent on recall-oriented retrieval plus letting the model rerank a short candidate list — **not** on graph expansion, which changes neither recall nor agent accuracy here.

Cost vs. benefit on-device (F3). Semantic retrieval's recall gain costs ~500× latency locally (median 47 s vs 0.09 s). A practical on-device agent therefore wants the *cheapest* path to high recall@K — a direction (hybrid lexical+semantic, cached embeddings, or a learned sparse retriever) this study motivates but does not yet resolve.

10. Limitations

- **Localization, not end-to-end resolution.** We measure where the agent *localizes* the fix, not whether it writes a passing patch; we do not execute patches (no Tier-B). Localization is necessary but not sufficient for issue resolution.
- **One bounded agent decision, E2B only.** The agent makes a single top-K choice with Gemma 4 **E2B**; we do not run E4B, a multi-turn repair loop, or tool use. These are natural extensions (§11), not claims made here.
- **Python-only graph** (stdlib `ast`); other languages via Tree-sitter are future work. SWE-bench Lite is Python-only, so this does not affect the reported results.
- **Bare-name call resolution** can over-connect same-named symbols, inflating graph edges; mitigated by confidence but not eliminated.
- **Small N (24 tasks)** — CIs are correspondingly wide; we report them rather than hide them.
- **Single machine / single run** for timing; latencies are indicative, not benchmarked.

11. Future Work

Since recall gates the agent (F4), the priority is cheaper high-recall retrieval: a combined ranker `S=αL+βE+γT+δG` with a **test-evidence term** `T`, hybrid lexical+semantic fusion, and budget-aware subgraph compression. On the agent side: Gemma 4 **E4B**, a multi-turn repair loop with tool use, and Tier-B sandboxed patch execution to measure end-to-end resolution. Add per-edge-type and multi-hop ablations.

12. Conclusion

Running the competition's own **Gemma 4 E2B locally on a 16 GB Mac**, we show that what helps a small agent localize bugs is retrieval **recall**: better recall@10 (semantic) lifts the agent's accuracy, and the model, reading ten candidates, nearly doubles top-1 localization over trusting the retriever's rank-1 — while a provenance-disciplined code graph is valuable for *auditability* but its lexically-seeded expansion changes neither retrieval nor agent accuracy, and semantic retrieval's recall costs ~500× latency. The actionable lesson for on-device SE agents is to spend the budget on cheap high-recall retrieval and let the model rerank, not on graph expansion. We report every outcome — positive, negative, and inconclusive — with confidence intervals, and treat that rigor, on real hardware with the mandated model, as the contribution.

References

1. C. S. Xia, Y. Deng, S. Dunn, L. Zhang. *Agentless: Demystifying LLM-based Software Engineering Agents*. arXiv:2407.01489, 2024 (FSE 2025).
2. C. E. Jimenez, J. Yang, et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024. arXiv:2310.06770.
3. J. Yang, C. E. Jimenez, et al. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. NeurIPS 2024. arXiv:2405.15793.
4. S. Ouyang, W. Yu, K. Ma, Z. Xiao, et al. *RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph*. ICLR 2025. arXiv:2410.14684.
5. F. Yamaguchi, N. Golde, D. Arp, K. Rieck. *Modeling and Discovering Vulnerabilities with Code Property Graphs*. IEEE S&P 2014.

6. P. Lewis, E. Perez, et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020. arXiv:2005.11401.
7. D. Edge, H. Trinh, et al. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. arXiv:2404.16130, 2024.
8. V. Karpukhin, B. Oğuz, et al. *Dense Passage Retrieval for Open-Domain Question Answering*. EMNLP 2020. arXiv:2004.04906.
9. S. Robertson, H. Zaragoza. *The Probabilistic Relevance Framework: BM25 and Beyond*. Found. Trends IR, 2009.
10. N. Reimers, I. Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP 2019. arXiv:1908.10084.
11. Z. Nussbaum, J. X. Morris, et al. *Nomic Embed: Training a Reproducible Long Context Text Embedder*. arXiv:2402.01613, 2024.
12. Gemma Team, Google DeepMind. *Gemma 4 Technical Report*. arXiv:2607.02770, 2026 (competition-provided base model family; E2B/E4B ≈ 2.3B/4.5B effective params). Model card: ai.google.dev/gemma/docs/core/model_card_4.

Reproducibility & released resources

The full implementation, the 24-task manifest, and the harness are released (Apache-2.0). `run_localization.py --semantic` + `evaluate.py` reproduce the retrieval table (§7); `run_gemma_localization.py --semantic` reproduces the Gemma results (§7.4, Table 2). Every graph edge carries provenance and confidence; unresolved targets are marked, not invented — the graph is auditable end to end.